

Übung 2: Asynchrone Programmierung (CompletableFuture)

1. Szenario: Das Smart-Home-System

Wir entwickeln die Steuerungssoftware für ein Smart Home. Sensoren liefern Daten (Temperatur, Helligkeit), Kameras machen Bilder und Aktoren (Licht, Heizung) werden gesteuert. Da Netzwerkzugriffe und Hardware-IO Zeit kosten, dürfen wir den Haupt-Thread nicht blockieren.

Wir nutzen `CompletableFuture` (eingeführt in Java 8, verbessert in Java 9+), um diese Aufgaben effizient und lesbar zu verketteten.

Basisaufgaben

Aufgabe 1: Ein einfacher Sensor (`supplyAsync`)

Simuliere einen Temperatursensor, der 1 Sekunde zum Messen braucht.

- Nutze `CompletableFuture.supplyAsync(...)`.
- Gib im Lambda 25.5 (Grad Celsius) zurück.
- Nutze `.get()` (nur hier zu Testzwecken!), um das Ergebnis in der `main` auszugeben.

Aufgabe 2: Weiterverarbeitung (`thenApply`)

Die Daten kommen oft im Rohformat. Erweitere die Kette aus Aufgabe 1:

- Hänge `.thenApply(...)` an, um den Wert (`Double`) in einen String umzuwandeln, z.B. 25.5 °C.
- Gib das Ergebnis aus.

Aufgabe 3: Konsumieren ohne Rückgabe (`thenAccept`)

Am Ende einer Kette wollen wir oft nur etwas ausgeben (Side Effect).

- Ersetze das `.get()` und `System.out.println` am Ende durch `.thenAccept(text -> System.out.println(text))`.
- WICHTIG: Damit das Programm nicht sofort beendet, hänge am Ende `.join()` an (oder nutze `Thread.sleep` in der `main`), da `supplyAsync` in einem Daemon-Thread läuft.

Aufgabe 4: Fehlerbehandlung (`exceptionally`)

Sensoren fallen gelegentlich auch aus.

- Kopiere den Code der vorherigen Aufgabe.
- Modifiziere `supplyAsync` so, dass es eine `RuntimeException("Sensorausfall")` wirft.
- Nutze `.exceptionally(...)`, um im Fehlerfall den Wert `Keine Daten` zurückzugeben.
- Prüfe, ob die Ausgabe entsprechend wechselt.

Aufgabe 5: Abhängige Schritte (`thenCompose`)

Manchmal brauchen wir das Ergebnis von Schritt A, um Schritt B zu starten (A → B).

- **Schritt A:** Lade die ID der aktuell aktiven Kamera aus einer Textdatei (z.B. `camera-id.txt` auf dem Desktop).
- **Schritt B:** Lade das Bild basierend auf dieser ID (z.B. `Cam-01.jpg` auf dem Desktop).
- Nutze `thenCompose`, um diese beiden asynchronen Operationen zu verketteten (FlatMap-Prinzip), sodass du am Ende *ein* Future mit dem Bild hast, nicht `Future<Future<File>>`.

- Beispielcode für den plattformunabhängigen Zugriff (funktioniert auch auf Windows, wenn die Dateien auf dem Desktop liegen):

```
// imports: java.nio.file.Files, java.nio.file.Path
Path desktop = Path.of(System.getProperty("user.home"), "Desktop");
Path camIdFile = desktop.resolve("camera-id.txt");
String camId = Files.readString(camIdFile).trim();

Path imageFile = desktop.resolve(camId + ".jpg"); // oder .png
byte[] imageBytes = Files.readAllBytes(imageFile);
System.out.println("Bildgröße: " + imageBytes.length + " Bytes");
```

Wichtig: Die Datei-IO in Schritt A und B soll jeweils asynchron in `supplyAsync` laufen und via `thenCompose` verkettet werden.

Aufgabe 6: Unabhängige Schritte kombinieren (thenCombine)

Wir wollen die Durchschnittstemperatur berechnen.

- Starte Task A: Liest den Wert aus `temp-wohnzimmer.txt` (z.B. 22.0) vom Desktop.
- Starte Task B: Liest den Wert aus `temp-kueche.txt` (z.B. 24.0) vom Desktop.
- Beide sollen **gleichzeitig** laufen (jeweils `supplyAsync`).
- Nutze `thenCombine`, um auf *beide* zu warten und den Durchschnitt $(val1 + val2) / 2$ zu berechnen.
- Beispiel-Code zum Lesen (plattformunabhängig, auch Windows):

```
// imports: java.nio.file.Files, java.nio.file.Path
private static double loadTemperature(String filename) {
    Path file = desktopPath().resolve(filename);
    try {
        return Double.parseDouble(Files.readString(file).trim());
    } catch (IOException e) {
        throw new UncheckedIOException("Konnte Temperatur nicht lesen: " + file, e);
    }
}
```

Aufgabe 7: Auf alle warten (allOf)

Wir verlassen das Haus und wollen alle Lichter ausschalten.

- Erstelle eine Liste von 3 Futures (Licht Wohnzimmer, Licht Küche, Licht Bad), die jeweils kurz “warten” und dann “Aus” zurückgeben.
- Nutze `CompletableFuture.allOf(...)`, um ein Future zu erhalten, das fertig ist, wenn *alle* Lichter aus sind.
- Gib danach “Alles dunkel” aus.

Zusatzaufgaben

Aufgabe 8: Timeouts (Java 9+)

Ein externer Wetterdienst antwortet manchmal nicht.

- Erstelle ein `CompletableFuture`, das 5 Sekunden schläft.
- Nutze `.orTimeout(1, TimeUnit.SECONDS)`, um nach 1 Sekunde abubrechen.
- Fange die `TimeoutException` ab (z.B. mit `exceptionally`) und gib “Wetterdienst nicht erreichbar” zurück.
- *Alternative:* Probiere `.completeOnTimeout("Standard-Wetter", 1, TimeUnit.SECONDS)` aus.

Aufgabe 9: Custom Executor

Standardmäßig nutzt `CompletableFuture` den `ForkJoinPool.commonPool()`. Das ist schlecht für IO-Aufgaben (blockierende Netzwerkanfragen), da der Pool klein ist.

- Erstelle einen eigenen Executor: `ExecutorService executor = Executors.newFixedThreadPool(10);`.
- Übergebe diesen Executor als zweites Argument an `supplyAsync`.
- Vergiss nicht, den Executor am Ende mit `executor.shutdown()` zu beenden.